

Sumit Tanaji Biranje

AI Engineer · LLM Engineer · Agentic AI Systems

+91 93268 53778 · sumitbiranje9@gmail.com · Pimpri-Chinchwad, Maharashtra · linkedin.com/in/sumit-biranje-

SKILLS

Agentic Systems

Multi-agent orchestration, MCP tools & servers, agent-as-tool, tool calling, planning & task routing, memory (short & long-term), guardrails, evaluation & observability

LLM Orchestration & Frameworks

LangChain, LangGraph, OpenAI Agents SDK, Google ADK, FastMCP, Pydantic structured output

Prompt Engineering

Zero / one / few-shot, dynamic, contextual, role-based, prompt chaining

RAG & Retrieval

Embeddings, semantic & hybrid search, re-ranking, graph-enhanced & vectorless RAG, chunking

Graph & Knowledge

Neo4j, FalkorDB, Cypher, ontology & knowledge-graph design, node traversal

Python & Backend

Python (sync & async, OOP), FastAPI, Uvicorn, REST APIs, asyncio / ThreadPoolExecutor, design patterns

Web, Data & Tools

HTML, CSS, JS, TypeScript, PHP, MySQL, Flask, Streamlit, DuckDB, tree-sitter, Git, GitLab, Postman, JIRA APIs, CANoe

AI-Assisted Development

Claude Code, OpenAI Codex, GitHub Copilot — coding & productivity assistants

EDUCATION

Diploma, Computer Science & Engineering

DKTE Yashwantrao Chavan Polytechnic, Ichalkaranji · 78.36%

SSC — Govindrao High School

Ichalkaranji · 72.40%

LANGUAGES

Marathi (Native)
Hindi (Conversational)
English (Professional)

SUMMARY

AI Engineer building AI-driven tools, multi-agent systems and production backend systems. Strong in LLM orchestration, agentic workflows, RAG pipelines, graph databases and advanced Python across sync and async patterns. Focused on practical AI systems with solid architecture, memory, safety and execution reliability — on a foundation of automotive software quality.

EXPERIENCE

Sr. Engineer — AI Engineering

KPIT Technologies, Pune

Apr 2025 — Present

- Build production AI tools and internal assistants — agentic workflows with custom MCP tooling, RAG systems and short- and long-term agent memory, on scalable FastAPI backends across sync and async Python.
- Ship with evaluation, observability and guardrails for reliable, cost-aware execution.

Infotainment System Test Engineer

KPIT Technologies, Pune

Jun 2023 — Mar 2025

- Authored functional test cases and performed L1 reviews of cases written by junior engineers, backed by strong software requirement specification (SRS) understanding.
- Ran functional, regression and integration testing for automotive infotainment software; among the highest defect loggers on the team, surfacing ad-hoc and exploratory defects beyond planned cases.
- Recognised with multiple customer and internal-management appreciations; unblocked critical internal features.

Trainee Engineer

KPIT Technologies, Pune

Jan 2022 — May 2023

- Interpreted requirement, screen-display and design documents to design test cases.
- Created single-function, interrupt-handling and multi-device-switching test cases.

Intern

KPIT Technologies, Pune

2021

- First exposure to automotive software validation and quality workflows.

KEY PROJECTS

- **AI-Assisted Regression Test Selection** — automatically decides which regression tests to run when a software build changes. Differs two builds, extracts the changed code entities, and traces their impact through a code knowledge graph (FalkorDB / Neo4j) via node traversal — with tree-sitter multi-language parsing and a crash-resilient, checkpointed pipeline.

Python · FalkorDB · Neo4j · tree-sitter · DuckDB · Streamlit

- **Code-to-Knowledge-Graph Pipeline** — builds the code knowledge graph that powers the regression test selector. Parses a multi-language codebase (Java, Kotlin, C, C++, XML, HAL) with tree-sitter and import resolvers — linking method definitions to their usages across files — then ingests nodes, properties and relationships into the graph, covering every layer from application and services through frameworks, ISP/BSP and HAL.

Python · tree-sitter · Neo4j / FalkorDB · multi-language parsing

- **Requirement-to-Code Traceability** — maps software requirements to the code that implements them. An agent explores a module's codebase and extracts its behaviour flow and sequence, then maps each requirement one-by-one to the relevant code elements — cross-referencing requirement (SRS) and design (SDS) specifications to resolve how internal and external components interact.

Python · multi-agent framework · SRS / SDS analysis

- **Multi-Agent Test Case Reviewer** — automatically reviews the quality of manually written test cases against guidelines and requirement traceability, producing structured feedback reports that replace slow peer review. Built with seven specialized review agents (OpenAI Agents SDK) and Pydantic-structured output, with parallel data ingestion.

Python · OpenAI Agents SDK · Pydantic · Streamlit

- **Automation Feasibility Analyzer** — evaluates whether manual test cases can be automated and outputs a per-test feasibility verdict with justification, helping teams prioritise automation effort. A config-driven LangChain pipeline (schema and criteria externalized to JSON) with Pydantic-validated output and a wizard-style Streamlit UI.

Python · LangChain · Pydantic · Streamlit

CURRENTLY EXPLORING

Active learning — not yet part of the core skills above.

- **Model fine-tuning** — LoRA / QLoRA and open-source model adaptation for domain-specific tasks.
- **Model serving & inference** — vLLM, quantization, and local / air-gapped inference with Ollama.
- **MLOps & deployment** — Docker, CI/CD, cloud, and monitoring for AI services.